

Optimal IPL Playing XI Selection and Batting Order Optimisation

Using Integer Programming and Goal Programming

1. Project in one sentence

This project uses historical IPL data to choose the best **Playing XI** from a candidate squad and place the selected players in the most suitable **batting order**. It solves the same cricket decision using two Operations Research methods:

1. **Integer Programming (IP)**: select the feasible XI and batting order with the highest overall predicted score.
2. **Goal Programming (GP)**: select the feasible XI and batting order that best meets several targets, such as batting strength, bowling depth, top-order quality, and recent form.

The result is a decision-support tool, not a claim that the model can perfectly predict a match. It gives a consistent, data-based recommendation under the chosen assumptions.

2. The real-world problem

An IPL franchise normally has a squad larger than 11 players. Before a match, the team management must answer two connected questions:

- Which 11 players should be in the playing team?
- In what order should those players bat?

This is difficult because a player can be valuable in different ways:

- A batter may have a high strike rate but poor recent form.
- A bowler may provide important bowling depth even if they are weak with the bat.
- An all-rounder contributes to both batting and bowling.
- An opener should generally be chosen for positions 1--3, while a finisher may be more suitable for positions 5--7.
- The team must include at least one wicketkeeper and enough bowling options.

A simple ranking of players is insufficient. For example, simply selecting the 11 players with the highest batting scores could select too few bowlers or no wicketkeeper. Optimisation chooses players **and** respects cricket constraints.

3. Project scope

Input

The user supplies or selects:

- Historical IPL ball-by-ball dataset.
- Candidate squad, for example 15--20 available players.
- Player roles: wicketkeeper, batter, all-rounder, bowler.
- Optional context: opponent, venue, season, or recent-form window.
- Model settings such as batting target, bowling target, and priority weights.

Output

The system returns:

- Selected Playing XI.
- Batting order from position 1 to 11.
- Role distribution: wicketkeepers, batters, all-rounders, bowling options.
- Total batting, bowling, form, and match-up scores.
- Constraint-satisfaction status.
- A comparison between the Integer Programming and Goal Programming solutions.

Scope for first review

For the first review, present the dataset, proposed features, variables, objectives, constraints, two model formulations, planned interface, and evaluation plan. Final numerical results are generated only after data cleaning and implementation.

4. Dataset and what it provides

Use a public IPL ball-by-ball dataset, preferably **Cricsheet IPL data** or a clearly identified Kaggle IPL dataset. The selected dataset must be confirmed as unique within the class group list.

Typical delivery-level columns include:

Field	Meaning	Project use
match_id, season, date	Identifies a match and season	Filter seasons / recent form
venue	Ground where the match occurred	Venue-specific analysis
batting_team, bowling_team	Teams in the innings	Opponent context
striker, non_striker	Batters at the crease	Batting statistics
bowler	Player bowling the ball	Bowling statistics

Field	Meaning	Project use
over, ball	Delivery position in innings	Powerplay, middle, death phases
batsman_runs, total_runs, extras	Runs from delivery	Strike rate, economy
is_wicket, dismissal_kind	Wicket information	Wickets and batting dismissals

Data-cleaning tasks

- Standardise player names across seasons.
- Remove duplicate rows.
- Treat wides and no-balls correctly while calculating balls faced and legal deliveries.
- Identify batting dismissals correctly; run-outs and retired hurt need careful treatment.
- Convert over numbers into phases:
 - Powerplay: overs 1--6.
 - Middle overs: 7--15.
 - Death overs: 16--20.
- Join a manually curated player-role table, because ball-by-ball data may not reliably provide roles such as wicketkeeper or all-rounder.

5. Statistics and scores derived from data

The raw delivery data must be converted to one row per player, or one row per player-position combination.

Batting measures

Measure	Formula / meaning
Runs	Total runs scored by a player
Balls faced	Legal balls faced, excluding wides
Batting average	<code>runs / dismissals</code>
Strike rate	<code>(runs / balls faced) x 100</code>
Boundary percentage	Proportion of scoring through fours and sixes
Dot-ball percentage	Proportion of balls with zero batter runs
Position performance	Historical runs / strike rate when batting at each position
Phase performance	Performance in powerplay, middle, and death overs

Bowling measures

Measure	Formula / meaning
Wickets	Dismissals credited to bowler
Legal balls bowled	Deliveries excluding wides and no-balls as appropriate
Economy rate	runs conceded / overs bowled
Bowling strike rate	balls bowled / wickets
Dot-ball percentage	Proportion of legal balls producing zero runs
Phase economy	Economy in powerplay, middle, and death overs

Other measures

- **Recent-form score:** performance over recent matches or a recent number of innings.
- **Opponent match-up score:** performance against a selected opponent, if sufficient observations exist.
- **Venue score:** performance at the chosen venue, if sufficient observations exist.
- **Role indicators:** known variables telling the model whether a player is a wicketkeeper, batter, all-rounder, or bowler.

Normalisation

Statistics have different units: strike rate may be around 100--180, while economy may be around 6--12. Before combining them, convert them to comparable scores, such as values from 0 to 100.

For a positive metric such as strike rate:

$$\text{Normalised score}_i = 100 \times \frac{v_i - \min(v)}{\max(v) - \min(v)}$$

For a metric where lower is better, such as economy rate:

$$\text{Normalised bowling score}_i = 100 \times \frac{\max(v) - v_i}{\max(v) - \min(v)}$$

6. Key terms used in the models

Term	Meaning in this project
Set	A collection, such as the candidate-player set P or batting-position set K
Parameter	An input value known before optimisation, such as a player score or role indicator
Decision variable	A value the solver must decide, such as whether a player is selected
Objective function	The mathematical measure to maximise or minimise

Term	Meaning in this project
Constraint	A rule that every valid solution must follow
Feasible solution	Any selection and batting order that satisfies all hard constraints
Optimal solution	The feasible solution that has the best objective value
Binary variable	A variable restricted to 0 or 1
Weight	A coefficient that states how important one component is relative to another
Target	A desired level used in Goal Programming
Deviation variable	A variable measuring how far a Goal Programming solution is below or above a target

7. Sets, indices, and parameters

Sets and indices

Symbol	Meaning
P	Set of all candidate players in the available squad
i	Index for a player, where $i \in P$
$K = \{1, 2, \dots, 11\}$	Set of batting positions
k	Index for batting position
WK	Set of wicketkeepers in the candidate squad
BAT	Set of specialist batters
AR	Set of all-rounders
TOP	Set of players eligible for top-order positions, typically 1--3

Parameters

Symbol	Meaning
R_{ik}	Estimated batting contribution of player i at position k
O_i	Normalised bowling-capability score of player i
F_i	Normalised recent-form score of player i
M_i	Opponent or venue match-up score of player i
q_i	1 if player i can bowl; 0 otherwise
λ	Weight placed on bowling strength in the IP model
T_B	Target team batting score for Goal Programming
T_O	Target total bowling score for Goal Programming

Symbol	Meaning
T_T	Target top-order batting score for Goal Programming
T_F	Target team recent-form score for Goal Programming
p_B, p_0, p_T, p_F	Goal priorities or penalty weights

8. Approach 1: Integer Programming

8.1 What is Integer Programming?

Integer Programming (IP) is an optimisation method in which one or more decision variables must take integer values. When a decision is yes/no, the variable is usually **binary**:

$$0 = \text{No}, \quad 1 = \text{Yes}$$

The IPL problem is naturally an IP problem because:

- A player cannot be selected as 0.4 of a player.
- A player is either in the Playing XI or not selected.
- A player is either assigned to batting position 3 or not assigned there.
- Each position must be filled by exactly one player.

If the variables were continuous, the model might create an invalid answer such as “select 0.6 of Player A and 0.4 of Player B.” Integer restrictions prevent this.

8.2 Decision variables

Selection variable

$$x_i = \begin{cases} 1, & \text{if candidate player } i \text{ is selected in the XI,} \\ 0, & \text{otherwise.} \end{cases}$$

Batting-position assignment variable

$$y_{ik} = \begin{cases} 1, & \text{if player } i \text{ bats at position } k, \\ 0, & \text{otherwise.} \end{cases}$$

Example

If Player A is selected and made opener:

$$x_A = 1, \quad y_{A,1} = 1$$

For all other positions of Player A:

$$y_{A,2} = y_{A,3} = \cdots = y_{A,11} = 0$$

If Player B is not selected:

$$x_B = 0, \quad y_{B,k} = 0 \quad \forall k$$

8.3 What does R_{ik} mean?

R_{ik} is not a decision; it is an input estimated from historical data. It represents the expected batting contribution of player i if placed at batting position k .

For example:

Player	Position 1	Position 3	Position 6
Specialist opener	88	64	40
Middle-order batter	52	82	66
Finisher	25	48	90

The model should prefer:

- the opener at position 1 or 2,
- the middle-order batter at position 3 or 4,
- the finisher at position 5--7.

A practical score can be built as:

$$R_{ik} = 0.40 \times \text{normalised average runs}_{ik} + 0.35 \times \text{normalised strike rate}_{ik} + 0.15 \times \text{recent form}_i$$

The exact weights are design choices. They should be justified and tested in sensitivity analysis.

8.4 Integer Programming objective function

The proposed objective is:

$$\max Z_1 = \sum_{i \in P} \sum_{k=1}^{11} R_{ik} y_{ik} + \lambda \sum_{i \in P} O_i x_i$$

First term: batting-order quality

$$\sum_{i \in P} \sum_{k=1}^{11} R_{ik} y_{ik}$$

This adds the expected score for every assigned player-position pair.

- If player i is not assigned to position k , then $y_{ik} = 0$, so that score adds nothing.
- If player i is assigned to position k , then $y_{ik} = 1$, so R_{ik} is included.

Thus, the model gives value not merely for selecting good players, but for placing them at suitable positions.

Second term: bowling depth

$$\lambda \sum_{i \in P} O_i x_i$$

This gives additional value to players with stronger bowling ability.

- O_i is the bowling score of player i .
- $x_i = 1$ only when the player is selected.
- λ controls the trade-off.

If λ is high, the model gives more importance to bowling. If it is low, the model focuses more strongly on batting-order score.

Interpretation

The IP model returns the feasible XI and batting order with the largest combined performance score under the chosen weights.

8.5 Integer Programming constraints

Constraint 1: Select exactly 11 players

$$\sum_{i \in P} x_i = 11$$

This prevents the solver from selecting fewer or more than 11 players.

Constraint 2: Each selected player gets exactly one batting position

$$\sum_{k=1}^{11} y_{ik} = x_i \quad \forall i \in P$$

How it works:

- If $x_i = 1$, the sum of assignments must be 1. The player receives one batting position.
- If $x_i = 0$, the sum must be 0. The player cannot receive a position.

This links player selection and batting-order assignment.

Constraint 3: Every batting position is filled by one player

$$\sum_{i \in P} y_{ik} = 1 \quad \forall k = 1, 2, \dots, 11$$

For each position k , exactly one player must be assigned. Therefore, the final batting order has no empty position and no duplicate player.

Constraint 4: At least one wicketkeeper

$$\sum_{i \in WK} x_i \geq 1$$

A valid cricket XI needs a wicketkeeper. WK is the set of candidate wicketkeepers.

Constraint 5: Minimum number of specialist batters

$$\sum_{i \in BAT} x_i \geq 4$$

This ensures the XI has a sufficient batting base. The number can be changed based on the group's cricketing assumption.

Constraint 6: Minimum number of all-rounders

$$\sum_{i \in AR} x_i \geq 1$$

This ensures at least one player contributes in both disciplines. This could be a hard rule or changed to a soft goal in Goal Programming.

Constraint 7: At least five bowling options

$$\sum_{i \in P} q_i x_i \geq 5$$

Here:

$$q_i = \begin{cases} 1, & \text{if player } i \text{ can bowl,} \\ 0, & \text{otherwise.} \end{cases}$$

A T20 innings has 20 overs, and any individual bowler can bowl a maximum of four overs. Five bowling options provide full coverage and flexibility. This is a simplified planning rule, not a replacement for a separate bowling-over allocation model.

Constraint 8: Top-order eligibility (optional but useful)

For positions 1, 2, and 3, restrict assignments to top-order-capable players:

$$\sum_{i \in TOP} y_{ik} = 1 \quad \forall k \in \{1, 2, 3\}$$

This prevents a tail-end specialist bowler from being assigned as an opener just because of noise or sparse statistics.

Constraint 9: Binary restrictions

$$x_i \in \{0, 1\}, \quad y_{ik} \in \{0, 1\}$$

These restrictions make the solution a valid discrete cricket line-up.

8.6 Small conceptual example

Suppose the candidate squad has 15 players. The solver considers all valid ways to:

- choose 11 out of 15,
- ensure the selected XI includes a wicketkeeper, enough batters, an all-rounder, and five bowlers,
- assign the selected 11 to positions 1--11,
- score every choice based on R_{ik} and bowling value.

It does not enumerate every option manually. A solver such as CBC through PuLP searches efficiently and returns the best feasible solution.

9. Approach 2: Goal Programming

9.1 What is Goal Programming?

Goal Programming (GP) is an extension of linear/integer programming used when there are several objectives or targets that matter simultaneously.

In normal Integer Programming, we write one main objective, such as:

max team score

But IPL selection has competing aims:

- high batting strength,
- sufficient bowling strength,
- a strong top order,
- good recent form,
- role balance.

A single weighted score may hide these trade-offs. Goal Programming instead says:

“Set desired targets for each important criterion and minimise the failure to meet them.”

The same decision variables x_i and y_{ik} are used. Therefore, Goal Programming solves the same Playing XI and batting-order problem, not a separate unrelated problem.

9.2 Deviation variables

For each target, GP introduces two non-negative variables:

$$d_g^- \geq 0, \quad d_g^+ \geq 0$$

- d_g^- : how much the result is **below** the target.
- d_g^+ : how much the result is **above** the target.

Example

Suppose the desired total batting score is 700.

$$\text{Actual batting score} + d_B^- - d_B^+ = 700$$

If the actual score is 650:

$$650 + d_B^- - d_B^+ = 700$$

Then:

$$d_B^- = 50, \quad d_B^+ = 0$$

The team is 50 points below the target.

If the actual score is 730:

$$730 + d_B^- - d_B^+ = 700$$

Then:

$$d_B^- = 0, \quad d_B^+ = 30$$

The team exceeds its batting target by 30 points.

For a “minimum desired strength” goal, only under-achievement d^- should be penalised. Exceeding the target is not a problem.

9.3 Goals for this project

Goal 1: Team batting strength

$$\sum_{i \in P} \sum_{k=1}^{11} R_{ik} y_{ik} + d_B^- - d_B^+ = T_B$$

- T_B is the desired total batting-order score.
- Penalise d_B^- because falling short is undesirable.

Goal 2: Bowling strength

$$\sum_{i \in P} O_i x_i + d_O^- - d_O^+ = T_O$$

- T_O is the target bowling score.
- Penalise d_O^- .

Goal 3: Top-order quality

$$\sum_{i \in P} \sum_{k=1}^3 R_{ik} y_{ik} + d_T^- - d_T^+ = T_T$$

This goal ensures the first three batting positions have sufficient expected quality.

Goal 4: Recent form

$$\sum_{i \in P} F_i x_i + d_F^- - d_F^+ = T_F$$

This avoids building a team entirely from career statistics while ignoring current performance.

9.4 Goal Programming objective

A weighted goal-programming objective can be written as:

$$\min Z_2 = p_B d_B^- + p_O d_O^- + p_T d_T^- + p_F d_F^-$$

The solver chooses the XI and batting order that minimise undesirable shortfalls across all the goals.

Meaning of priority weights

Weight	Meaning
p_B	Importance of meeting batting target
p_O	Importance of meeting bowling target
p_T	Importance of meeting top-order target
p_F	Importance of meeting recent-form target

If batting is the main concern, use a larger p_B . If the pitch is expected to favour bowlers or the opponent has a very strong batting side, increase p_O .

Two ways to define priority are common:

1. **Weighted GP:** numeric weights are used, for example $p_B = 5$, $p_O = 4$, $p_T = 3$, $p_F = 2$.
2. **Pre-emptive (lexicographic) GP:** the first priority is satisfied as much as possible before considering the next priority. For a student project, weighted GP is simpler to implement and explain.

9.5 Hard constraints still apply

Goal Programming does not remove core cricket-validity rules. The following remain hard constraints:

$$\sum_{i \in P} x_i = 11$$

$$\sum_{k=1}^{11} y_{ik} = x_i \quad \forall i \in P$$

$$\sum_{i \in P} y_{ik} = 1 \quad \forall k$$

$$\sum_{i \in WK} x_i \geq 1$$

$$\sum_{i \in P} q_i x_i \geq 5$$

and all binary restrictions.

Hard vs soft constraints

Type	Meaning	Example
Hard constraint	Must never be violated	Exactly 11 players; at least one wicketkeeper
Soft goal	Desirable but can be missed at a measurable cost	Achieve batting score of 700

This distinction is central to Goal Programming.

10. Integer Programming vs Goal Programming

Aspect	Integer Programming	Goal Programming
Same decision problem?	Yes: XI selection and batting order	Yes: XI selection and batting order
Main idea	Maximise a combined performance score	Minimise shortfall from multiple targets
Objective	One objective function	Several goals represented through deviations
Player selection	Binary variables x_i	Same binary variables x_i
Batting order	Binary assignment variables y_{ik}	Same binary assignment variables y_{ik}
Role rules	Hard constraints	Same hard constraints
Best use	When one overall score is acceptable	When management wants explicit balance between several priorities
Output	Maximum-score feasible XI	Most target-balanced feasible XI

Why two answers can differ

The IP model may select a very high-scoring batting unit with just enough bowling capability because the weighted objective values batting highly. The GP model may select a slightly lower-batting-score XI if that helps meet a bowling target, top-order target, and recent-form target more consistently.

Neither answer is automatically “correct” in every situation. The project compares the trade-off transparently.

11. End-to-end solution workflow

Step 1: Obtain and load data

- Download the public IPL ball-by-ball dataset.
- Store it in CSV/JSON format.
- Load it into Python using Pandas.

Step 2: Clean and validate data

- Check missing columns and duplicate rows.
- Standardise player names.
- Identify legal deliveries, wickets, and batting dismissals correctly.
- Filter the seasons or matches needed for analysis.

Step 3: Create a role table

Maintain a small curated table like this:

Player	Role	Can bowl?	Top-order eligible?
Player A	Wicketkeeper-batter	0	1
Player B	Batter	0	1
Player C	Batting all-rounder	1	1
Player D	Bowler	1	0

This is necessary because role definitions cannot always be derived reliably from scorecard data alone.

Step 4: Compute player features

Calculate batting and bowling metrics. Build position-specific historical performance R_{ik} . If a player has sparse history in one position, use a fallback such as:

- broader role average,
- league average for that role and position,

- smoothed average combining player and league data.

Step 5: Normalise and score

Transform metrics to a common scale and create:

- R_{ik} : batting-position score,
- O_i : bowling score,
- F_i : recent-form score,
- optionally M_i : match-up score.

Step 6: Run the Integer Programming model

- Set λ .
- Solve using PuLP/CBC, Pyomo, or OR-Tools.
- Return the highest-objective feasible XI and batting order.

Step 7: Run the Goal Programming model

- Set targets T_B , T_O , T_T , T_F .
- Set priority weights.
- Solve with the same hard constraints.
- Return the line-up with minimum weighted shortfall.

Step 8: Compare results

Display:

- Common players in both XIs.
- Players selected only by IP or only by GP.
- Batting-order changes.
- Each model's batting, bowling, form, and top-order score.
- Goal shortfalls from GP.

Step 9: Perform sensitivity analysis

Change one input at a time:

- λ in IP,
- targets in GP,
- goal priority weights,
- minimum number of bowlers,
- recent-form period.

Then observe whether the selected XI remains stable or changes significantly.

12. Example scenario for explaining in viva

Assume there are 16 available players. The squad includes:

- 2 wicketkeepers,
- 6 specialist batters,
- 4 all-rounders,
- 4 specialist bowlers.

The model must select 11 players, at least one wicketkeeper, at least four batters, at least one all-rounder, and at least five players who can bowl.

Integer Programming result

The IP model may select the XI with the greatest total score:

- Strong opening pair,
- high strike-rate middle order,
- one wicketkeeper,
- five available bowlers,
- a batting order based on historical position suitability.

Goal Programming result

Suppose the IP team narrowly misses the preferred bowling target. GP may replace a pure batter with an all-rounder or bowler, reducing batting score slightly but reducing the bowling-target shortfall substantially.

Interpretation

- IP answers: “Which valid XI has the highest total weighted score?”
- GP answers: “Which valid XI best achieves our collection of planned targets?”

13. User interface plan

A Streamlit dashboard can contain the following pages or sections:

A. Load Data

- Upload CSV/JSON or select the provided IPL dataset.
- Show number of matches, deliveries, seasons, teams, and players.

B. Statistics

- Batting leaderboard.
- Bowling leaderboard.
- Player detail page.
- Batting-position score table.
- Role distribution in candidate squad.

C. Optimisation Settings

- Choose candidate squad.
- Choose opponent and venue if implemented.
- Set minimum role requirements.
- Choose λ for the IP model.
- Set GP targets and priority weights.

D. Run Optimisation

- Button: Run Integer Programming.
- Button: Run Goal Programming.

E. Results

- Selected XI table.
- Batting order table.
- Constraint checks.
- Objective value for IP.
- Goal deviations for GP.
- Side-by-side model comparison.

14. Tools

Task	Suggested tools
Data handling	Python, Pandas, NumPy
Optimisation	PuLP with CBC, Pyomo, or Google OR-Tools
Visualisation	Matplotlib, Plotly
User interface	Streamlit
Development	Jupyter Notebook, VS Code/Cursor, GitHub

Recommended implementation choice

For a final-year OR course project, use:

- **Pandas** for feature generation,
- **PuLP** for both Integer Programming and weighted Goal Programming,
- **Streamlit** for the user interface.

This is straightforward to explain, quick to prototype, and sufficient for the project requirements.

15. Assumptions and limitations

Be explicit about assumptions in the presentation and report.

Assumptions

- Historical performance is a useful proxy for expected future performance.
- Player roles are provided through a curated role table.
- Scores are estimated from available data and are not exact match predictions.
- All listed squad players are available and fit for the match.
- The model recommends a pre-match batting order; in real cricket, innings circumstances can require dynamic changes.

Limitations

- Ball-by-ball datasets may not include injuries, training performance, player fatigue, leadership, or dressing-room factors.
- Some players have sparse data at certain batting positions.
- IPL teams can have overseas-player restrictions; add this later if the squad contains domestic and overseas labels.
- A full bowling-over sequence is intentionally outside the core two-model scope. It can be added as future work.

16. Future scope

- Add overseas-player constraint: maximum four overseas players in a Playing XI.
- Add bowling-over allocation as a third IP model.
- Include player auction cost or salary-cap constraints.
- Include opposition-specific bowler-versus-batter match-ups.
- Build a predictive ML layer to estimate R_{ik} rather than using score-based aggregation only.
- Add robust optimisation for uncertain form or injury risk.

- Compare recommendations against actual match XIs and evaluate retrospectively.

17. Viva questions and concise answers

Why is this an Integer Programming problem?

Because player selection and batting-position assignment are discrete yes/no decisions. A player must be selected or not selected, and must be assigned to one position or not assigned. Fractional players or fractional positions are invalid.

What is the objective in the IP model?

Maximise expected batting-position contribution plus selected bowling strength, subject to a valid Playing XI and batting-order constraints.

Why use assignment variables y_{ik} ?

They allow the model to decide not only who plays, but where each selected player bats. They connect a player's historical suitability to a specific batting position.

What is the difference between an objective and a constraint?

The objective defines what the model wants to optimise. Constraints define rules that cannot be violated. For example, maximise expected score is an objective; select exactly 11 players is a constraint.

Why is Goal Programming needed if IP already gives the best score?

IP optimises one combined score. Goal Programming explicitly handles several competing targets and shows the cost of not meeting each target, such as shortage in bowling strength or top-order quality.

What do d^- and d^+ represent?

They represent under-achievement and over-achievement of a target. For a minimum batting goal, d^- measures how much the chosen team is below the desired batting target.

Are the two models solving the same problem?

Yes. Both choose a feasible Playing XI and batting order from the same squad. They differ only in how they evaluate quality: one maximises a total score, while the other minimises deviations from multiple targets.

How will you evaluate the models?

By checking feasibility, role balance, objective/goal values, stability under parameter changes, and retrospective comparison with historical selections or performance where possible.

18. Final presentation statement

We formulate IPL Playing XI selection and batting-order assignment as a single discrete optimisation problem. Integer Programming identifies the feasible line-up with the highest overall expected score. Goal Programming solves the same problem while explicitly balancing targets for batting strength, bowling depth, top-order quality, and recent form. The system uses historical IPL ball-by-ball data, provides statistics through an interface, and allows users to compare both optimisation-based recommendations.